

AI Guardrail — Agent Governance Layer: Design & Module Plan

Status: §7.1 (the AI Guardrail side) is implemented and tested — 116 tests passing, verified live end-to-end against the real DPDP Engine. §7.2/§9 (the VOXA/Kotlin side) is not built by this project; this document is VOXA's integration reference. §4.1 reflects the exact contract as implemented, not a proposal.

Companion inputs this design is grounded in:

- Internal call transcript, Mahesh Ramakrishnan / Mausam Ghosh, 2026-09-20 (agent identity approaches, the agent-metadata vs. activity-log split, registration options).
- GOVERNANCE_LAYER_MOVEMENT_PLAN.md (D:\PoC_Workspaces\AI_DPDP_Engine) — the DPDP Engine's own explicit scope-split document, addressed directly to this project.
- Direct code investigation of D:\PoC_Workspaces\AI-Governance-version2 (existing audit/alarm/auth building blocks) and D:\PoC_Workspaces\JIO-VOXA-version-1 (the first real external agent this layer will serve).

1. Scope split (reaffirmed, unchanged from the DPDP Engine's own plan)

Concern	Lives in
Notices, consents, consent-lifecycle decisions, DPDP-specific compliance events	DPDP Engine — unchanged, pure consent manager, nothing agent-specific
Agent identity / registry, invoking-user tracking, general Activity Log, agent-governance compliance events, Stats	AI Guardrail — new Agent Governance Layer (this document)

The "Decision Passthrough" module below is **not** a second consent engine. It is an identity-verified front door in front of the existing, unmodified `dpdp_client.check_decision(...)` call. The DPDP Engine remains the sole authority on "is there consent for this" — this layer never re-derives that answer.

2. Data model

2.1 Agent Registry (static — one row per registered agent, written once at registration)

Field	Notes
<code>agent_id</code>	Server-generated UUID. The technical identity.
<code>agent_name</code>	Human-readable label, e.g. "VOXA — Swiggy Skill".
<code>agent_secret_hash</code>	Hashed at rest, never stored or logged in plaintext.
<code>business_unit</code>	Org unit that owns this agent.
<code>owner_name</code>	The person/team accountable for this agent — distinct from <code>business_unit</code> , this is a real contact, not a label.
<code>location_of_deployment</code>	e.g. <code>on-device:android</code> , or a server region/hostname for a cloud-deployed agent.
<code>in_house_or_external</code>	Flag.
<code>identity_assignment_timestamp</code>	Set once, at registration.
<code>status</code>	<code>active</code> / <code>revoked</code> — minimal lifecycle only; the fuller life-cycle enum discussed on the call (<code>created/deployed/paused/etc.</code>) is explicitly deferred, not needed for this POC.

Deliberately excluded from this table: any per-user field. See 2.2 for why.

2.2 Per-call signature → Activity Log row (dynamic — one row per call, append-only)

```
{
  "ts": "2026-09-20T10:00:00Z",
  "agent_id": "...",
  "invoking_user_id": "...",
  "action": "...",
  "outcome": "SERVED | BLOCKED",
  "reason_code": "...",
  "device_id": "...",
  "latency_ms": 0,
  "correlation_id": "..."
}
```

`reason_code` was added after a real case where a BLOCKED row couldn't answer "why" on its own — tracing the reason meant separately cross-referencing Compliance Events (for a DPDP DENY) or, for a content-pipeline block via `/guardrail_validate`, nowhere durable at all. It's populated at both existing log-write sites (the DPDP decision's `reason_code`, or `/guardrail_validate`'s `flag_reason`) and is null for an ordinary SERVED row with nothing to explain. A BLOCKED/PAYMENT_DECLINED outcome from `/guardrail_validate` now also raises a Governance Compliance Event (AGENT_POLICY_VIOLATION), mirroring the DPDP-denial path's existing AGENT_UNAUTHORIZED_ACTION, so content-pipeline blocks show up in Compliance Events/Stats the same way DPDP denials always have.

`device_id` was added for the same reason as `reason_code` -- a decision-check row and its paired activity row (same `correlation_id`) could each carry a `device_id`, but neither was recorded anywhere durable, so there was no way to cross-check them after the fact. Populated from the same field on both `/v1/agent/decisions/check` and `/v1/agent/activity`'s requests. Confirmed by VOXA's own code inspection (not assumed): a single persisted UUID, generated once per install and read from one SharedPreferences key, is the **only** value their `deviceId()` function can ever return -- the exact same call is used in both `registerAgent()` and `checkDecisionFor()`, for every skill. So this value is guaranteed to be identical across Swiggy/Teams/Yahoo Finance/Kite on one install, and identical between a decision-check row and its paired activity row -- a mismatch would indicate a real bug, not a legitimate different device.

`invoking_user_id` lives here, not in the registry, because one agent can be used by many different people over time (or, for a single-user app like VOXA, by whatever stable pseudonym represents that one account/device — see §6). Baking a single user identity into the static registry would be wrong by construction the moment an agent serves more than one person.

No raw request/response content is ever logged here — `action` is a short operation label (e.g. "swiggy_cart_update"), not the user's message. See §5 for the reasoning.

External validation of this split: this mirrors Alexa Skills Kit's own architecture — a static, per-skill **Skill ID** assigned once at registration, versus a separate **User ID** (and Device ID) carried on every individual request, deliberately scoped so the same identity fields never get collapsed into one. Amazon's own design treats "which skill" and "who's talking to it right now" as two different identifiers for exactly this reason.

2.3 Governance Compliance Events (agent-scoped, append-only)

Same **shape** as the DPDP Engine's own `compliance_events` table (event type, version, severity, subject, structured categories, source, reason code, correlation id, two timestamps) but the subject is an `agent_id`, not a DPDP `principal_ref`. Suggested event types, parallel to DPDP's own vocabulary: AGENT_IDENTITY_UNVERIFIED, AGENT_UNAUTHORIZED_ACTION, AGENT_POLICY_VIOLATION, AGENT_SCOPE_EXCEEDED. Raised automatically by the Identity Verification and Decision Passthrough modules on failure — never sent by the calling app itself.

2.4 Stats

No new store. Pure read-side aggregation over 2.1–2.3 (calls by outcome, calls by agent, events by severity, distinct agents/users seen) — same approach as the DPDP Engine's own `GET /v1/stats` (a GROUP BY query, no running counters maintained separately).

3. Identity issuance & verification flow

1 Registration (once per install/deployment): POST /v1/agent/register — self-registration

(transcript's Option 3: as good as CI/CD-driven registration, avoids the "owner forgets to wire in the ID" weakness of manual registration and the ID-proliferation risk of auto-assign-on-first-call). Request carries agent_name, business_unit, owner_name, location_of_deployment, in_house_or_external. Response: agent_id, a freshly generated **agent-specific** secret (not a shared token — a shared secret is explicitly called out, in the DPDP Engine's own movement plan, as "the anti-pattern for agent identity: one shared secret can't tell agents apart"), and identity_assignment_timestamp.

1 Every subsequent call carries X-Agent-Id + the issued secret. Verification happens inline,

as part of the same request (not a separate pre-flight "verify identity" round trip) — unknown agent_id, wrong secret, or status=revoked → reject immediately with **401** (see §4.1 for the exact error shape), before the request reaches the decision or logging logic at all. Only a verified identity's request is processed.

1 Open item, not resolved here: reinstall/re-registration behavior. Either accept some identity

churn (a reinstall becomes a new agent_id), or design an idempotent registration key — needs an explicit decision, not an assumption.

1 Decided for this PoC: secret loss with no reinstall (app data cleared, storage corruption, etc.)

is treated the same as reinstall — accept identity churn, register as a new agent. There is no rotate/reissue endpoint; agent_auth.py only exposes register_agent/verify_agent, and governance_db.revoke_agent only flips status, it never reissues a secret. A secure reissue path needs its own bootstrap credential (you can't gate "give me a new secret" behind the secret that was just lost) — real scope, not worth building for a PoC. Revisit for a production version, most likely as an admin-mediated reissue (admin_pii/super_admin-gated, logged as its own governance event) rather than a self-service endpoint.

4. New API surface (AI Guardrail's own, distinct from anything DPDP-labeled)

Route	Purpose
POST /v1/agent/register	One-time registration (§3).
POST /v1/agent/decisions/check	Identity-verified passthrough to dpdp_client.check_decision(...); logs the outcome to the Activity Log; raises a Governance Compliance Event on identity failure or a DPDP DENY/UNKNOWN.
POST /v1/agent/activity	Lightweight, non-gating activity log entry for calls that don't need a consent decision at all (e.g. a read operation).
GET /v1/agent/* (admin)	List/browse endpoints backing the four new Admin UI tabs.

Deliberately namespaced under /v1/agent/..., distinct from the DPDP Engine's own /v1/decisions/check etc., so nobody integrating against either service confuses the two contracts.

4.1 Exact wire contract (as implemented — api.py)

Auth headers, every route except /register: X-Agent-Id: <agent_id> + Authorization: Bearer <secret>. Not a body field.

POST /v1/agent/register — no auth required (this is how it's obtained). Request: {agent_name, business_unit, owner_name, location_of_deployment, in_house_or_external}. Response (200): {"agent_id": "...", "agent_secret": "...", "identity_assignment_timestamp": "...". The secret's JSON key is agent_secret, shown exactly once, here.

POST /v1/agent/decisions/check. Request: {principal_ref, purpose, operation, data_categories, recipient_ref?, policy_context?, correlation_id?, device_id?}. Response (200): dpdp_client.check_decision(...)'s return value plus an echoed correlation_id: {"decision",

"guard_failed", "error", "decision_id", "notice_version", "reason_code", "correlation_id"}. correlation_id is optional in the request — **if omitted, the server generates one and echoes it back** (not left to dpdp_client.py's own internal fallback, so the same id ties together the DPDP Engine call, this layer's activity/event rows, and the response the caller sees). This endpoint has **no latency_ms field** — only /activity captures that.

POST /v1/agent/activity (non-gating). Request: {invoking_user_id, action, outcome, latency_ms?, correlation_id?, reason_code?, device_id?}. Response (200): {"status": "logged", "correlation_id": "..."} (same omit-then-generate-and-echo rule). ts is always server-computed on receipt — there is no client-timestamp field. latency_ms is optional and entirely client-supplied and unvalidated; the Guardrail does not compute it, so the timer start point is the calling agent's own choice to define. reason_code is likewise entirely caller-supplied, taken at face value — this endpoint never calls DPDP and runs no check of its own, so Guardrail has no independent way to know why a self-reported BLOCKED outcome happened. Send it whenever outcome=BLOCKED, or the row is opaque after the fact (found in practice: a self-reported swiggy_payment_initiation outcome of BLOCKED with no reason_code and no way to tell it apart from a DPDP-side denial, even though the paired decision_check for the same purpose had actually returned ALLOW — two different questions, easy to conflate without an explanation attached). Reuse the same correlation_id as any /v1/agent/decisions/check call this activity is reporting the outcome of, so the two rows can be told apart from unrelated ones sharing the same agent_id.

Error shape on identity failure: always 401 (403 is not used), FastAPI's default HTTPException body: {"detail": "Agent identity could not be verified."}. Unknown agent_id, wrong secret, and a revoked agent all produce this **same** generic message — verify_agent() returns None uniformly for all three failure modes by design, so there is currently no machine-readable way to tell them apart. This is a deliberate choice, not an oversight: not confirming whether a given agent_id exists at all is a reasonable security posture (comparable to a login form that doesn't confirm whether a username exists). Revisit only if a concrete need for finer-grained client-side error handling shows up.

A genuine network failure (Guardrail unreachable/timeout) never produces this shape at all — no HTTP response comes back — so a caller can already distinguish "identity rejected" (got a 401 with this body) from "couldn't reach the Guardrail" (no response) without any additional signal needed.

5. Activity logging content policy

No raw request/response content, ever. This isn't a new rule invented for this layer — it's the same discipline already established everywhere in this project: audit_logger.py never logs raw input (only masked text or a hash), the Guardrail Activity panel deliberately never shows message content, and the DPDP Engine's own compliance events outright reject raw_prompt/message/text fields. If proof of *what* happened is ever needed without storing it, use a content_hash (SHA-256), the same pattern the DPDP Engine already uses. action is a short operation label, not a message body.

6. Ingress-validation principle (revised — see rationale below)

Not every message gets validated externally. Routing every keyed-in/spoken utterance through a network call to this Guardrail *before* an on-device LLM even processes it would defeat the actual purpose of running the LLM on-device: it would make the assistant's basic operation depend on network + Guardrail uptime, add a round-trip's latency to every turn, and — the sharper point — ship 100% of raw conversational content off the device by default, which is a *worse* privacy posture than keeping it on-device, not a better one.

The corrected rule: validate only at the point of external effect. A network call to this layer happens only when an agent is about to take an action that mutates state outside the device (a cart write, a payment) — never merely because the user said something. And even then, what crosses the network is operation *metadata* (agent id, purpose, operation, data category, correlation id) — never the raw utterance or tool arguments, per §5.

This also matches why the classic "injection guard protects the platform" justification is weaker for a single-user, on-device agent than for the original AI-Governance chatbot: that system's LLM has too_l_broker access to *other customers'* records, so a hijacked prompt could exfiltrate someone else's data. An on-device assistant like VOXA only ever acts within one user's own linked accounts — there's no other tenant's data to protect by screening every utterance over the network.

7. Module-wise development plan

7.1 AI Guardrail side (built and tested first)

Module	Responsibility
Agent Registry	Schema (§2.1) + POST <code>/v1/agent/register</code> .
Identity Verification	Inline dependency/middleware checked on every <code>/v1/agent/*</code> call before anything else runs.
Decision Passthrough	Identity-verified wrapper around the existing <code>dmdp_client.check_decision(...)</code> — no changes to <code>dmdp_client.py</code> itself. New purposes (e.g. <code>voxa_swiggy_cart_management</code> , <code>voxa_swiggy_payment_initiation</code>) get registered on the DPDP Engine side, the same way <code>device_diagnosis_ai_analysis</code> was for JioCare Helper — a DPDP-Engine-side task, not this layer's.
Activity Log	Append-only store; written by the Decision Passthrough module (every check, any outcome) and by the lighter POST <code>/v1/agent/activity</code> for non-gated reads.
Governance Compliance Events	Raised automatically on identity failure or DPDP denial — never sent directly by a calling app.
Stats	Pure aggregation, no new writes (§2.4).
Admin UI	One new top-level button in <code>App.jsx</code> (mirrors the existing flat nav: Chat Bot / Admin Configuration / Analytics / Testing Dashboard), opening a new page with its own 4-tab bar (Registered Agents / Activity Log / Compliance Events / Stats), following <code>AdminConfig.jsx</code> 's existing <code>tab+table+Refresh</code> pattern exactly.

7.2 VOXA side (spec to hand off once 7.1 is built and tested — not built by this project)

Module	Responsibility
<code>AiGuardrailClient.kt</code>	New client, same shape as the existing <code>SwiggyMcpClient/KiteClient</code> .
Registration at install	In <code>MainActivity.onCreate</code> , alongside where <code>swiggyAuth/kiteAuth</code> are already constructed — register once, persist <code>agent_id</code> + secret via <code>EncryptedSharedPreferences</code> , reusing the exact storage pattern already proven for <code>swiggy_auth</code> .
Subject pseudonym	New state VOXA doesn't have today — confirmed by direct investigation, VOXA has no end-user identity of any kind. Generate a stable per-install/per-account UUID once, persist it the same way, use it as <code>principal_ref</code> for DPDP consent checks.
Write-operation gating	Inside <code>SwiggyMcpClient.callTool()</code> : for <code>update_food_cart</code> (confirmed existing) and a future payment-initiation tool (not yet present in the app), call POST <code>/v1/agent/decisions/check</code> *before* the real Swiggy MCP call; block on anything but <code>ALLOW</code> . Read operations (<code>search_restaurants</code> , <code>get_restaurant_menu</code> , <code>get_food_cart</code> , <code>get_addresses</code>) pass straight through, optionally with a non-gating POST <code>/v1/agent/activity</code> call.
Activity logging hook	The existing <code>onToolExecuted</code> callback is already the right attachment point — no restructuring needed.

8. Worked example: VOXA / Swiggy (grounded in direct code investigation, not assumption)

Confirmed scope for this VOXA integration: Swiggy and Yahoo Finance only. Kite and Teams are present in the VOXA codebase but explicitly excluded from this plan.

Confirmed from D:\PoC_Workspaces\JIO-VOXA-version-1:

- On-device Gemma via LiteRT-LM, genuine LLM tool-calling (not a sentinel-token protocol), one shared Conversation/tool registry across Swiggy/Kite/Yahoo Finance/Teams — no per-skill sandboxing at the model level.
- Swiggy calls go through Swiggy's own MCP server (<https://mcp.swiggy.com/food>, OAuth 2.1 + PKCE) via a single choke point, `SwiggyMcpClient.callTool()`.
- Reads: `get_addresses`, `search_restaurants`, `get_restaurant_menu`, `get_food_cart`. Write: `update_food_cart` only — **no checkout/payment call exists in the app today**; a Settings-screen disclaimer explicitly tells the user to check out in the real Swiggy app.
- No phone number, no payment/card data, no GPS anywhere in the Swiggy code path (the manifest doesn't even request location permission).
- Zero existing consent/PII/toxicity/injection logic anywhere in the codebase — full-tree grep, zero hits. This is genuinely new construction on VOXA's side, not an extension of a partial system.

Recommended consent surface for Swiggy: gate only `update_food_cart` (and, once it exists, a payment-initiation tool) — not the reads, since none of them touch anything beyond commercial/public data or an address tag. Suggested purpose/operation/category, reusing already-established DPDP vocabulary rather than inventing new names: `purpose=voxa_swiggy_cart_management`, `operation=UPDATE_CART`, `data_categories=["ORDER_DATA"]`; for a future payment tool, `purpose=voxa_swiggy_payment_initiation`, `operation=INITIATE_PAYMENT`, `data_categories=["PAYMENT_TOKEN"]` — the same operation/category names already proven working in this project's own payment-gate integration.

9. Per-skill hand-off checklist for VOXA (Swiggy + Yahoo Finance)

Each VOXA skill gets its **own** Agent Registry entry and its own `agent_id/secret` — per the Alexa Skills Kit precedent (§2.2): every skill is a separately identified thing in that model too, regardless of whether it happens to process personal data. What differs sharply between these two skills is **what gets sent to the Guardrail and when** — one has a real DPDP consent surface, the other has none at all, confirmed by direct code investigation, not assumption.

9.1 Skill profile comparison

	Swiggy	Yahoo Finance
Personal/user data involved	Yes — address tag, cart contents, order data	None — public market data only, no account, no credentials (YahooFinanceClient.kt's own doc comment: <i>"no account, no credentials involved anywhere here"</i>)
Write operations	Yes — <code>update_food_cart</code> (+ a future payment tool)	No — <code>search</code> and <code>get_quote</code> only, both read-only
Needs a DPDP consent-check call	Yes , on the write path only	No — there is no Data Principal data being processed, so there is no purpose to register and nothing to gate
Needs an Agent Registry entry	Yes	Yes — for identity/traceability, not for consent

	Swiggy	Yahoo Finance
Needs Activity Log entries	Yes, every tool call	Yes, every tool call — pure observability (is the gateway up, how often is it called), not privacy-driven
Can raise a Governance Compliance Event	Yes — AGENT_IDENTITY_UNVERIFIED (bad credentials) or AGENT_UNAUTHORIZED_ACTION/AGENT_POLICY_VIOLATION (a real DPDP DENY)	Only AGENT_IDENTITY_UNVERIFIED — the data-related event types can never fire here, since no decision-check call exists for this skill at all

9.2 Registration (both skills, done once each, same mechanism, §3)

VOXA registers **two** separate agents at first run, not one for "VOXA" generally:

- 1 agent_name="VOXA – Swiggy Skill", plus business_unit, owner_name, location_of_deployment="on-device:android", in_house_or_external.
- 2 agent_name="VOXA – Yahoo Finance Skill", same other fields.

Persist each returned agent_id + secret separately (e.g. two new EncryptedSharedPreferences keys, mirroring the existing "swiggy_auth" naming convention already used in the app — this is a pattern VOXA's own code already establishes, not something new being introduced).

One subject pseudonym, shared across both skills — not one per skill. This is a deliberate departure from the Alexa comparison, worth stating explicitly rather than silently copying the pattern: Alexa scopes userId differently per skill specifically to stop two *unrelated third-party developers* from correlating the same person across their independent skills. That threat doesn't exist here — both skills are the same app, same team, same device. The DPDP Engine already isolates consent per (principal_ref, purpose), not per principal_ref alone (proven by the JioCare Helper integration: *"a second, hypothetical purpose does not inherit this grant... consent_records is keyed on (principal_ref, purpose)"*) — so reusing one pseudonym across skills doesn't leak anything between them; the purpose field already provides that isolation. One persisted UUID per install is simpler for VOXA to build than N, with no real privacy cost.

9.3 Swiggy — what goes, when

Reads (search_restaurants, get_restaurant_menu, get_food_cart, get_addresses):

- No consent-check call — there's no write/legal-basis question for a read.
- Optional, non-gating: POST /v1/agent/activity **after** the Swiggy call completes (never blocks it),

with agent_id=Swiggy, invoking_user_id=the shared pseudonym, action="swiggy_read:<tool_name>", outcome=SERVED.

- Never send: raw address text, menu contents, cart items, prices, the Swiggy OAuth token.

Writes (update_food_cart, and a future payment-initiation tool):

- POST /v1/agent/decisions/check **before** the real Swiggy MCP call, with agent_id=Swiggy, principal_ref=the shared pseudonym, purpose="voxa_swiggy_cart_management", operation="UPDATE_CART", data_categories=["ORDER_DATA"] (for a future payment tool: purpose="voxa_swiggy_payment_initiation", operation="INITIATE_PAYMENT", data_categories=["PAYMENT_TOKEN"]).

- Anything but ALLOW → block the Swiggy call entirely; do not call SwiggyMcpClient.callTool() for that write.

- ALLOW → proceed with the real Swiggy call, then log a SERVED activity entry.

- Never send: cart contents, item names/prices, the OAuth token, the raw user utterance.

- Exact insertion points: `SwiggyTools.kt`'s `addItemToCart` (the tool function that calls `update_food_cart`) and the auto-add-winner path inside `findBestRestaurantForDish` — both currently call `SwiggyMcpClient.callTool("update_food_cart", ...)` directly; the decision-check call needs to sit immediately before that call in both places.

9.4 Yahoo Finance — *what goes, when*

- Only one model-facing tool, `getStockPrice`, which itself calls two read-only MCP methods (`search`, `get_quote`) — confirmed no personal data anywhere in this path.
- No consent-check call, ever** — there is no Data Principal data involved, so there is nothing to register a DPDP purpose for and nothing to gate. Do not invent a purpose for this just for consistency with Swiggy; that would be registering a consent requirement where none exists.
- Recommended, non-gating: `POST /v1/agent/activity` after each lookup, with `agent_id=Yahoo Finance`, `action="stock_price_lookup"`, `outcome=SERVED` (or `BLOCKED`-equivalent only on a network/timeout error) — purely operational monitoring (is the public gateway reachable), not privacy-driven.
- Never send: the actual company/ticker query text or the returned price data — not because either is sensitive (it isn't), but to keep one uniform "never send content, only a short action label" rule across every skill rather than carving out a content-is-fine-here special case.
- Exact insertion point: `YahooFinanceTools.kt`'s `getStockPrice` function, at its existing `onToolExecuted` callback (already called once on the error-path and once on the success-path) — that's already the natural, existing hook to attach the activity-log call to.

9.5 What travels on every call, summarized

Field	Scope	Notes
X-Agent-Id + secret	Per skill	Different for Swiggy vs. Yahoo Finance calls
<code>invoking_user_id / principal_ref</code>	Per install (shared)	One pseudonym, reused across both skills — see §9.2
<code>correlation_id</code>	Per request	Fresh every call, never reused
Message/tool-argument content	Never sent to <code>/v1/agent/decisions/check</code>	No skill's raw content crosses that boundary — metadata only, per §5. See §9.6 for the one deliberate exception (Teams, via a different endpoint)

9.6 Teams — *content validation (resolved after a real leak, not a hypothetical)*

Teams was originally out of scope for this document (§8: "Kite and Teams are present in the VOXA codebase but explicitly excluded from this plan"), but VOXA built a Teams skill anyway and gated its `sendTeamsMessage` with `purpose="voxa_teams_message_post"`, `operation="POST_MESSAGE"`, `data_categories=["MESSAGE_CONTENT"]` — a real `/v1/agent/decisions/check` call, same shape as Swiggy's write path. That call can only ever answer "is this agent allowed to post *any* message" — it never receives the message text, so it cannot tell "the meeting code is 43345" apart from "the weather is nice today." This is not a bug in that endpoint; it's §6's ingress-validation principle working exactly as designed for content that never needed to leave the device — except a Teams post **is** an external effect (real third parties see it immediately, unlike a structured Swiggy cart write), so metadata-only gating was the wrong fit for this specific case. This was confirmed as a real leak, not a theoretical gap, when a meeting code was posted to Teams unmasked.

Resolution: before calling `sendTeamsMessage`, VOXA also calls `POST /guardrail_validate` with the raw message text, and posts the returned `masked_output` to Teams instead of the raw text — never the original. This uses the same `correlation_id` as the paired `/v1/agent/decisions/check` call, so both sides of the same send (the purpose-gate

and the content-mask) can be tied together after the fact. `/guardrail_validate` already runs the existing masking/toxicity/payment-intent pipeline (`$api.py guardrail_validate`) — no new endpoint was needed for this.

Update — item 1 below is now resolved. `/guardrail_validate` now optionally accepts `X-Agent-Id` + `Authorization: Bearer <secret>` (still fully reachable without them — JioCare Helper and the Chat Bot page keep working unauthenticated) and a `correlation_id` in the request body (generated and echoed back if omitted, same rule as `/v1/agent/*`). When identity verifies, the outcome is written to the Agent Governance Layer's Activity Log as `content_check:<flag>` (SERVED/BLOCKED), under the same `agent_id` and `correlation_id` as the paired `/v1/agent/decisions/check` call — closing the exact gap found when a real Teams masking event showed up in `governance_audit.json` and the in-memory Guardrail Activity buffer, but nowhere under the Teams agent's own Activity Log. VOXA needs to start sending these two headers plus its existing `correlation_id` on its `/guardrail_validate` call for this to take effect on their side.

One thing this resolution does not yet close, tracked separately, not blocking this fix:

- 1 This closes the specific gap that caused the confirmed leak — a new `pii_rules.json` rule, `VERIFICATION_CODE` (meeting code / OTP / verification code / passcode / access code + a 4–8 digit number, under the `AUTHENTICATION` category), plus the pre-existing built-in `PHONE` recognizer, which already covers a plain phone number in text. It is **not** a general content-safety guarantee — it's regex-based pattern matching, not an exhaustive secret detector. Treat each new leak pattern found as its own rule to add, not evidence the mechanism itself is complete.

10. Open items requiring an explicit decision before implementation

- 1 Reinstall/re-registration behavior (§3.3) — accept identity churn, or build idempotent registration?
- 2 VOXA's subject-pseudonym mechanism (§7.2, §9.2) is genuinely new work on VOXA's side — needs their commitment, not just this layer's API contract.

- 1 Hard-block-unregistered-agents policy (transcript's Option 4a) is an organizational decision, not a technical default — out of scope for this design until explicitly requested.

- 1 Exact DPDP Engine purpose registration for the two Swiggy purposes above is a DPDP-Engine-side task to schedule separately. Yahoo Finance needs no such registration at all (§9.4).

- 1 Secret loss without reinstall (app data cleared, storage corruption) — **decided for this PoC:**

treated identically to reinstall, i.e. accept churn, register as a new agent. There is no rotate/reissue endpoint (`agent_auth.py` only exposes `register_agent/verify_agent`; `governance_db.revoke_agent` only flips status, it never reissues a secret). A secure reissue path needs its own bootstrap credential — real scope, not worth it for a PoC. Revisit for production, most likely as an admin-mediated reissue (`admin_pii/super_admin`-gated, its own governance event) rather than a self-service endpoint.

- 1 Self-deregistration — **not built, by design, not just by omission.** An agent can self-register (§3.1 — no auth needed, that's how identity is obtained in the first place) but cannot deregister or revoke itself; `POST /v1/agent/agents/{agent_id}/revoke` is admin-only (`admin_pii/super_admin` RBAC via `X-Role`), never callable with an agent's own `X-Agent-Id/secret`. Same asymmetry as item 5: identity `*creation*` is self-service, anything that removes an identity's ability to act is an admin decision. If a skill needs a clean way to retire itself, that's a new admin-mediated flow, not a change to the registration contract.

11. Sequencing

Build and test 7.1 (AI Guardrail's own governance layer) first, end-to-end, against a stub/manual caller. Only once that contract is stable does VOXA build 7.2 against it — so both sides can verify they generate/consume the same `agent_id/metadata` shape from a working reference, not a moving target.

12. Future identity model: Agent → Registration → Device (not built — target design only)

Status: proposed target design, captured here for VOXA's planning, zero code changes made for this section.

Nothing below is implemented, and per an explicit decision on this thread, **nothing that would require an existing VOXA install to reinstall gets built right now** — this section exists so VOXA can plan future builds against a stable target rather than the model shifting again later.

12.1 What's already correct and needs no change

Re-stated here because a review of this design mistook these for gaps — they aren't; they're already built exactly this way:

- **agent_id is already immutable and separate from the display name.** agent_id (server-generated

UUID) is the identity; agent_name (e.g. "VOXA — Swiggy Skill") is a human label only (§2.1).

- **Owner vs. invoking principal is already split**, deliberately, with the Alexa Skills Kit precedent

as justification (§2.2): owner_name (static, per-agent, "who's accountable") lives in the registry; invoking_user_id (dynamic, per-call, "who it's acting for right now") lives in the Activity Log. These must never collapse into one field — see §2.2's reasoning.

- **identity_assignment_timestamp already exists**, just labeled "Registered" in the current admin UI.

12.2 What's changing: a registrationId layer

The current model conflates "which skill" with "which install" — one agent_id *is* one install, which is exactly why reinstall forces a new identity (§10 item 1). The target model separates these:

```
AGENT (agt_voxa_swiggy_001)          -- stable, one per skill, no secret of its own
■
■■■■ REGISTRATION (reg_001) -- own secret, one per install/enrollment
■      ■■■■ DEVICE (dev_A)  -- correlation only, see 12.3
■■■■ REGISTRATION (reg_002) -- own secret
■      ■■■■ DEVICE (dev_B)
■■■■ REGISTRATION (reg_003) -- own secret
      ■■■■ DEVICE (dev_C)
```

agent_id becomes a stable, secret-free grouping label (identifies the *skill*, not the install). registration_id becomes the actual credentialed entity — each registration gets its own independent secret. This is a genuine improvement over both the current model (no grouping at all, one secret = one install = full identity) and a naively-shared-secret alternative (one secret shared across every install of a skill, where a single leak compromises every device running that skill worldwide): scoping the secret to registration_id means a leaked credential only ever compromises that one registration.

Identity chain and what each layer answers:

Identity	Answers	Status
agentId	Which agent/skill?	Already built (as the sole identity today; becomes a grouping label)
registrationId	Which deployment/registration of that agent?	New — the real target change
principalId	Which user is it acting for right now?	Already built (invoking_user_id/principal_ref)
deviceId	Which physical/runtime device?	Blocked — see 12.3
agentSignature	Can the *skill codebase* be cryptographically authenticated?	Blocked — see 12.4
deviceSignature	Can the *specific hardware* be cryptographically authenticated?	Blocked — see 12.4

Implementing `registrationId` requires VOXA's registration call to change shape (distinguishing "first registration of a new agent" from "another registration under an existing `agent_id`"), which is itself a VOXA app change — so, per the reinstall freeze above, this is a planning target, not current work.

12.3 `deviceId` — *aspirational, blocked on a real device fingerprint*

`deviceId` only adds information beyond `registrationId` if it's a fingerprint that survives an app reinstall on the same physical device. Nothing proposed so far gives it that: if it's just a freshly generated UUID at registration time, it's indistinguishable from `registration_id` and adds nothing. Modern Android has no reliable, permission-free way to get a stable hardware ID — `ANDROID_ID` resets on factory reset and now varies per app-signing key, and IMEI requires privileged permissions Google restricts. This is the same "genuinely new work on VOXA's side" gap already flagged in §7.2/§9.2 for the subject pseudonym — this section doesn't solve it, it just names the dependency. **Do not build `deviceId` until VOXA can commit to a real mechanism**; until then, treat `registration_id` as the only per-install identifier that exists.

Decided for this PoC: not required. A two-device scenario (same VOXA app, two different phones, two different people) was worked through explicitly and needs no `deviceId` to function correctly — `agent_id` and `principal_ref` are already generated independently per install, so two devices naturally get fully independent identities and consent state with zero collision risk. The only thing `deviceId` would still add is surviving a *reinstall on the same device*, which is out of scope per the reinstall freeze above. Not building this for the PoC.

12.4 `agentSignature` / `deviceSignature` — *two separate future initiatives, not fields*

Both names imply real cryptographic verification, and each is its own scoped project:

- `agentSignature` (authenticate *which skill codebase* is calling) would mean verifying something like the APK's code-signing certificate hash server-side — a code-provenance-verification subsystem.
- `deviceSignature` (authenticate *the specific hardware*) would mean integrating hardware

attestation, e.g. Google's Play Integrity API — a full external-service integration with its own failure modes (quota, outages, key rotation), not a value generated locally.

Neither should be scheduled as part of the `registrationId` work above. If either is wanted, it needs its own design doc and its own decision to invest, separate from this identity-model change.

Decided for this PoC: not required. Both are full standalone security subsystems (code-signing verification, hardware attestation via an external API) with their own infrastructure and failure modes — real scope for a production hardening pass, not this PoC. Left as future items only.

12.5 `agentVersion` — *accepted, low-risk, no reinstall required*

Worth adding for future registrations: no schema conflict, doesn't affect existing agents, doesn't force a reinstall (an install that never sends it just omits the field). Before adding it, decide what governance actually *does* with it — a version that's collected but never enforced against policy is a display-only field, not a governance control. Left as a future addition, not scheduled here.

13. Mocked IAM, Entitlements, and Incidents — implemented

Grounded in a real planning discussion (Mahesh's data architect, 2026-09-21): identity (§3) and consent (§1) are not the only governance questions. A third, separate one — **is this agent even scoped to attempt this at all** — sits *before* consent, answers a different question, and needed its own mechanism. Canonical example from that discussion: an agent authorized only for Swiggy-shaped actions attempting a payment on a completely different app is an **incident** (a security/scope violation), not an ordinary consent denial.

13.1 *Why this is a separate table, not a field on the Agent Registry*

The natural-seeming shortcut — store "allowed apps" directly on the agent's row at registration time — was considered and rejected. Two reasons: (1) an `agent_id` like "VOXA — Kite Skill" already *is*, by construction, permanently scoped to Kite

(§9.2) — asking "can this agent act as Kite" is tautological; the real question the scenario asks is "is this **user** allowed to use Kite at all," which is a fact about the `principal_ref`, not the already-registered agent identity. (2) Storing a mutable authorization fact on a mostly-static identity record creates two sources of truth the moment an independent "IAM sync" process needs to update it — the same reasoning that already keeps `activity_log/governance_events` separate from agents. So: `iam_entitlements` is its own table, checked live on every call (same "real-time, never cached" principle the architect insisted on for consent), never duplicated onto the Agent Registry.

13.2 Data model

```
iam_entitlements(principal_ref, agent_id, device_id, allowed_app, granted_at)
```

The lookup key is the three-part "signature" from that discussion — `*user + agent + the device/server hosting it*` — not `agent_id` alone. `device_id` is nullable and treated as a wildcard when NULL ("not yet restricted by device"), since VOXA didn't send `device_id` on every call until this feature existed — entitlements can be tightened to a specific device once real values are flowing.

`allowed_app` is derived from purpose via a small substring map (`api.py`'s `_purpose_to_app`): "swiggy", "teams", "kite", "yahoo". **Deliberately scoped, not universal**: a purpose that doesn't match any known app (e.g. this repo's own pre-existing BILLING_SUPPORT/AUTO_PAY purposes, unrelated to VOXA) skips the IAM gate entirely and proceeds straight to DPDP, exactly as before this feature existed. This was a real regression caught by the existing test suite during implementation — the first version fail-closed on `*any*` unmapped purpose, which incorrectly blocked every non-VOXA caller. IAM here answers a VOXA-specific scoping question; it is not a blanket policy over every purpose in the system.

No entitlement is currently granted for "kite", on purpose — that's the gap Scenario 5 depends on.

13.3 Wire contract

`/v1/agent/decisions/check`'s request gains `device_id` (optional; a missing value only matches an entitlement row whose own `device_id` is NULL). The check runs immediately after identity verification and `*before*` `dpdp_client.check_decision` is ever called — a scope violation never reaches DPDP at all, since it isn't a consent question.

13.4 Incidents — a new, higher tier than Compliance Events

```
incidents(incident_id, event_type, severity, agent_id, principal_ref, reason_code, correlation_id, created_at)
```

`incident_id` is a dummy string (INC-#####) — this never calls a real ITSM tool. Per the same discussion: **not every block is an incident**. No consent given, withdrawn consent, masked-but-passed content — all ordinary enforcement, alert-only at most, unchanged (Activity Log + `reason_code` + existing Alarms). `incidents` is reserved specifically for scope/authorization violations (AGENT_SCOPE_EXCEEDED today), raised only from the IAM-check failure path.

On creation: log the Activity Log row (BLOCKED, `reason_code`="IAM_SCOPE_EXCEEDED") → create the incident → attempt a notification email (`notifications.py`'s `send_incident_email`, reusing the same `notification_subscribers` list every other alarm email already reads from `pii_rules.json` — no new recipient config). Fire-and-log: a notification failure (e.g. SMTP_USER/SMTP_PASSWORD unset) never affects the already-decided DENY, same discipline as every other alarm path in this project. Post-incident investigation is explicitly out of scope, per the same discussion: the incident record just needs enough detail to be handed to a real system, not an investigation workflow of its own.

13.5 Admin UI

Two new tabs under Agent Governance: **Entitlements** (read-only — seeded/managed by whoever owns the IAM sync process, no grant/revoke UI yet) and **Incidents**. Ordered by category, not by build order: Registered Agents, Entitlements, Activity Log, Compliance Events, Incidents, Stats — the two identity/ authorization "setup" tabs grouped first, the three event-stream tabs next, Stats last as the aggregate over everything.

13.6 What VOXA needs to change

- 1 Register Kite as a real agent (`agent_name`="VOXA — Kite Skill") — identity must genuinely succeed;

this is an authorization test, not an identity-failure test.

1 Wire Kite's existing write action(s) to call `/v1/agent/decisions/check` first, same insertion pattern already used in `SwiggyTools.kt`.

1 Send `device_id` on **every** `/v1/agent/decisions/check` call going forward, not just at registration — the same value already generated and persisted locally.

1 Nothing else — the DENY response is kept in the same shape as any other DPDP-driven denial, so existing "not allowed" handling already covers it.

Guardrail deliberately will not grant Kite any entitlement — identity passes, the new IAM layer is what stops it.